

Инженеринг на софтуерните изисквания

Ивайло Андреев + Deepseek V3

20 юни 2026

Съдържание

1	Същност на Инженеринга на софтуерните изисквания – цел и особености	2
1.1	Цел и източници на изисквания	2
1.2	Основни етапи на Инженеринга на изискванията	3
2	Класификация на софтуерните изисквания	5
2.1	Йерархичен граф на изискванията	5
2.2	Същност на класификационните категории	5
3	Техники, прилагани за извличане, анализ и валидиране	7
3.1	Техники за извличане	7
3.2	Техники за анализ	9
3.3	Техники за валидиране	9
4	Специфициране и модели на софтуерните изисквания	10
4.1	Нива на документиране	10
4.2	Модели в зависимост от перспективата на системата	11
4.2.1	1. Перспектива "Данни"(Data perspective)	11
4.2.2	2. Перспектива "Функции"(Functional perspective)	12
4.2.3	3. Перспектива "Поведение"(Behavioral perspective)	12

1 Същност на Инженеринга на софтуерните изисквания – цел и особености

1.1 Цел и източници на изисквания

Основната цел на **Инженеринга на софтуерните изисквания (ИСИ)** е да дефинира, систематизира, документира и поддържа софтуерните изисквания по начин, който гарантира съответствие между крайната софтуерна система и реалните потребности на бизнеса и потребителите. Тази цел се постига чрез прилагане на систематични, повтаряеми и количествено измерими подходи, заимствани от традиционното инженерство, но адаптирани към спецификите на софтуерното развитие. Изискванията не съществуват във вакуум – те се извличат и съгласуват от широк спектър от източници, които могат да бъдат категоризирани според тяхната природа и степен на формалност:

- **Разговори със заинтересовани лица (ЗЛ):** Това представлява основният и най-богат източник на информация. Заинтересованите лица обхващат всички страни, които имат дял или интерес в софтуерния продукт – клиенти, крайни потребители, продуктови мениджъри, инвеститори, спонсори на проекта, регулаторни органи и дори конкуренти. Ефективното идентифициране и ангажиране на всички категории ЗЛ е критичен фактор за успеха на целия процес по извличане на изисквания.
- **Регулаторни изисквания:** Съвременният софтуер функционира в сложна правна и нормативна среда. Този източник включва национални и международни закони, секторни директиви, индустриални регламенти и вътрешни корпоративни политики, които налагат задължителни ограничения върху функционалността, сигурността, съхраняването на данни и поверителността на информацията.
- **Подобни системи:** Анализът на съществуващи софтуерни продукти, както конкурентни, така и наследени (*legacy systems*), функциониращи в организацията, предоставя ценни прозрения. Този анализ позволява идентифициране на утвърдени практики, често срещани функционални шаблони, както и на пропуски и недостатъци в съществуващите решения, които новата система би могла да преодолее.
- **Стандарти:** Международните и индустриални стандарти (ISO/IEC, IEEE, W3C) дефинират общоприети норми за качество, сигурност, оперативна съвместимост, преносимост и документация. Спазването на тези стандарти не само повишава качеството на продукта, но и улеснява интеграцията му с други системи и неговата сертификация.
- **Експерти от приложната област:** Специалистите с дълбоки познания в конкретния бизнес сектор (*domain experts*) притежават имплицитни знания за семантиката на процесите, бизнес правилата и конвенциите, които не са изрично документирани никъде. Техният експертен опит е незаменим при формулирането на изисквания от приложната област.

1.2 Основни етапи на Инженеринга на изискванията

Процесът по управление и изграждане на изискванията не следва строго линеен модел, а притежава силно изразен **итеративен и инкрементален характер**. Това се дължи на естеството на софтуерното инженерство, където разбирането на проблема и неговото решение се задълбочава и прецизира с течение на времето. Всеки етап от жизнения цикъл на изискванията може да бъде повторен многократно, като резултатите от един етап служат като вход за следващите, но и обратно – нови знания, придобити по време на анализ или валидиране, могат да наложат преразглеждане на по-ранни етапи.

Таблица 1: Етапи на жизнения цикъл на софтуерните изисквания

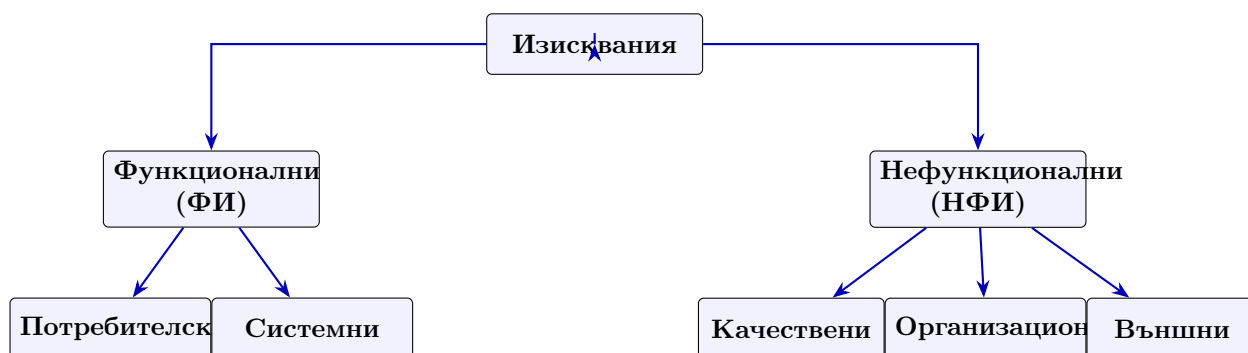
Етап	Характеристика и същност на етапа
0. Анализ за осъществимост	Еднократен процес , който се провежда в инициращата фаза на проекта. Той представлява предварителна оценка на няколко критични измерения: <i>техническа осъществимост</i> (наличие ли са необходимите технологии и компетенции), <i>финансова оправданост</i> (очакваната възвръщаемост на инвестицията оправдава ли разходите по разработка), <i>правна и регулаторна съвместимост</i> и <i>стратегическа съвместимост</i> с бизнес целите на организацията.
1. Извличане на изисквания	Това е активният процес на събиране, откриване, идентифициране и дефиниране на потребностите на заинтересованите лица. Той не представлява пасивно приемане на вече формулирани изисквания, а систематично проучване, което включва задаване на въпроси, наблюдение, прототипиране и други техники. Извличането има за цел да разкрие както явните (изрично изразените), така и скритите, имплицитни изисквания, за които заинтересованите лица може дори да не осъзнават, че съществуват.
2. Анализ на изисквания	Етап, посветен на структуриране, прецизиране и осигуряване на техническото качество на извлечените данни. Той включва дейности по категоризиране, приоритизиране, разрешаване на конфликти между противоречиви изисквания и моделиране. В рамките на този етап се прилагат строги критерии за качество – всяко изискване трябва да бъде еднозначно (допускащо само една интерпретация), пълно (съдържащо цялата информация, необходима за неговата реализация) и без вътрешни и външни противоречия (непротиворечащо на останалите изисквания и на бизнес логиката).
3. Документиране	Формално записване, структуриране и представяне на изискванията в подходящ медиум. Това може да включва естествен език, структуриран табличен формат, псевдокод и полуформални модели. Крайният резултат от този етап, след многократни итерации, е създаването на Спецификация на софтуерните изисквания (<i>SRS</i>), която служи като официален документ за комуникация между всички участници в проекта – клиенти, анализатори, архитекти, разработчици и тестери.
4. Валидиране	Това е проверка на най-високо ниво, която има за цел да установи дали съставеният документ и изградените модели реално отразяват и удовлетворяват истинските изисквания на заинтересованите лица. Валидирането задава фундаменталния въпрос: "Изграждаме ли правилния продукт?" За разлика от верификацията, която проверява техническата коректност ("Изграждаме ли продукта правилно?"), валидирането се фокусира върху стойността на продукта за потребителя и бизнеса.
5. Управление на изисквания	Този етап се базира на фундаменталното правило на софтуерното инженерство, че "промяната е единствената константа" . Тъй като пазарът, технологиите, регулациите, конкурентната среда и разбирането на потребителските нужди се променят непрекъснато, изискванията не могат да бъдат фиксирани веднъж завинаги. Този етап управлява целия жизнен цикъл на промените – от регистриране на заявка за промяна, през анализ на нейното въздействие, оценка на усилията и ресурсите, до приемане, отхвърляне или отлагане на промяната, както и проследяване на нейното изпълнение.

2 Класификация на софтуерните изисквания

Софтуерните изисквания се разделят глобално на бизнес и системни изисквания, но основната им таксономия се базира на функционалното им разделение, нивата на абстракция и произхода на изискването. Тази класификация е от съществено значение за правилното разбиране, документиране и управление на изискванията през целия им жизнен цикъл.

2.1 Йерархичен граф на изискванията

По-долу е представен структурният граф на изискванията, който илюстрира йерархичната им организация. Пунктираните линии и специфичното позициониране на Изискванията от приложната област (ИПО) визуализират тяхното специфично свойство да се припокриват с останалите категории в зависимост от методологията на анализа, избраната перспектива и конкретния контекст на разглеждане.



Фигура 1: Йерархична класификация на софтуерните изисквания

2.2 Същност на класификационните категории

- **Функционални изисквания (ФИ):** Тези изисквания описват *какво* точно трябва да прави системата. Те дефинират множеството от услуги, които системата предоставя, нейните реакции към специфични входни данни и вътрешни събития, както и нейното поведение в различни операционни ситуации. Функционалните изисквания определят същностната стойност на софтуерния продукт и са пряко свързани с бизнес процесите, които системата подпомага или автоматизира.
 - **Потребителски изисквания:** Дефинират функционалността на високо ниво на естествен език, често придружен от диаграми и скици. Те са предназначени за не-техническите заинтересовани лица – клиенти, мениджъри, бизнес анализатори и спонсори на проекта. Тези изисквания описват системата от гледна точка на нейните потребители и техните цели, без да навлизат в технически детайли за реализацията.
 - **Системни изисквания:** Това представлява детайлизирана, прецизирана и технически ориентирана версия на потребителските изисквания. Те описват софтуерните функции на ниво модули и компоненти, архитектурните детайли, интерфейсите между компонентите, трансформацията на данни и специфичните алгоритми. Системните изисквания служат като

основа за архитектурния дизайн, детайлното проектиране и имплементацията от страна на разработчиците.

- **Нефункционални изисквания (НФИ):** Докато функционалните изисквания дефинират *какво* прави системата, нефункционалните изисквания поставят ограничения върху *как* системата изпълнява своите функции. Те често се наричат "изисквания за качество" или "ограничения тъй като те определят критериите за оценка на операционните характеристики на софтуера. НФИ са толкова критични за успеха на системата, колкото и функционалните изисквания, тъй като те определят потребителското възприятие за качество, надеждност и полезност на продукта.
- **Качествени изисквания:** Дефинират операционните характеристики на софтуера, които са пряко свързани с неговото изпълнение в експлоатационна среда. Те включват:
 - * *Производителност:* Време за отговор, пропускателна способност, латентност, използване на системни ресурси.
 - * *Надеждност:* Готовност (достъпност), устойчивост на грешки, възможност за автоматично възстановяване, средно време между откази.
 - * *Сигурност:* Поверителност на данните, цялостност, автентичност, оторизация, одитируемост, устойчивост на атаки.
 - * *Използваемост:* Ефективност на взаимодействието, обучаемост, достъпност за хора с увреждания, естетика на интерфейса.
 - * *Поддръжка:* Модулност на архитектурата, повторна използваемост на компонентите, леснота на анализ, диагностика и извършване на промени.
- **Организационни изисквания:** Тези изисквания произтичат от вътрешните политики, стандарти и процедури на организацията, която разработва или експлоатира софтуера. Те включват стандарти за кодиране, дефинирани от организацията, задължителни езици за програмиране и технологии, изисквания към документацията, методики за разработка (например задължително използване на Agile или Waterfall), вътрешни правила за сигурност на информацията, корпоративни стандарти за потребителски интерфейс и изисквания за съвместимост с вътрешните информационни системи на организацията.
- **Външни изисквания:** Тези изисквания са наложени от фактори извън контрола на организацията разработчик или клиент. Те включват:
 - * *Законодателни изисквания:* Национални и международни закони, регулаторни директиви и правни норми, които системата трябва да спазва.
 - * *Изисквания за оперативна съвместимост:* Необходимост за взаимодействие с външни системи, държавни платформи, стандартизирани протоколи и формати за обмен на данни.
 - * *Етични изисквания:* Принципи за справедливост, недискриминация, прозрачност и защита на личните данни, които са част от обществените и корпоративния етос.

* *Географски и културни изисквания*: Поддръжка на различни езици, културни норми, местни стандарти за измерване и финансови конвенции.

- **Изисквания от приложната област (ИПО)**: Тези изисквания произтичат директно от предметната област на приложението (домейна), в която софтуерът ще функционира. Особеност е, че ИПО не представляват самостоятелна, изолирана категория, а се припокриват и пресичат с функционалните и нефункционалните изисквания. Всяко Изискване от приложната област, разгледано самостоятелно, представлява или функционално изискване (например бизнес правило за изчисление на данък, логика за обработка на медицински запис, алгоритъм за кредитен скоринг), или нефункционално ограничение (например изискване за съответствие със секторен стандарт, регулаторна норма в дадена индустрия, медицински протокол за сигурност на пациентски данни). Тяхното класифициране като функционални или нефункционални зависи изцяло от конкретния аналитичен контекст, избраната перспектива и институционалната рамка на анализа. ИПО имат особено силно припокриване с категорията на външните изисквания, особено когато става въпрос за секторни регулации и индустриални стандарти, които произтичат от спецификата на приложната област, но са външно наложени на организацията.

3 Техники, прилагани за извличане, анализ и валидиране

Процесът на инженеринг на изискванията се поддържа от богат набор от техники и методи, които се прилагат на различните етапи от жизнения цикъл на изискванията. Тези техники варират по своята формалност, мащабируемост, изисквания за ресурси и приложимост в различни контексти и типове проекти.

3.1 Техники за извличане

Извличането на изисквания представлява фундаменталният етап, в който се събира първичната информация от заинтересованите лица и други източници. Ефективността на този етап до голяма степен определя качеството на всички последващи дейности.

- **Интервюта**: Това е една от най-разпространените и гъвкави техники за извличане. Интервютата се провеждат в две основни форми:
 - *Отворени интервюта*: Представяват свободни, неструктурирани дискусии без предварително фиксиран въпросник. Те позволяват на интервюираното лице да разкрие информация, която не би била уловена при по-строги формати, и дават възможност за задълбочено проучване на неочаквани теми. Основният недостатък е по-голямата трудност при структуриране и сравняване на резултатите от различни интервюта.
 - *Затворени интервюта*: Провеждат се по предварително дефиниран въпросник със стандартизирани въпроси. Този формат улеснява сравняването на отговорите от различни респонденти, осигурява по-голяма покриваемост на предварително идентифицираните теми и намалява влиянието на интервюиращия върху получените отговори.

- **Анкетите:** Тази техника позволява мащабно събиране на количествени и качествени данни от географски разпръснати или многобройни групи заинтересовани лица. Анкетите са особено ефективни, когато е необходимо да се получи обратна връзка от голям брой потребители за приоритети, нагласи, предпочитания и честота на използване на определени функции. Стандартизираният формат на анкетите позволява статистическа обработка на резултатите, но ограничава дълбочината на получената информация.
- **Срещи със заинтересованите лица:** Това са съвместни сесии, които събират представители на различни групи заинтересовани лица за интерактивно обсъждане и моделиране на изискванията:
 - *Работни срещи (Workshops):* Интензивни, фасилитирани сесии, в които инженери по изискванията и потребители работят съвместно за извличане, анализиране и приоритизиране на изисквания. Тези срещи са особено ефективни за бързо постигане на консенсус и разрешаване на конфликти.
 - *Фокус групи:* Структурирани дискусии с малка група представители на целевата потребителска група, насочени към изследване на нагласите, възприятията и очакванията спрямо концепцията на софтуерния продукт.
 - *Мозъчна атака (Brainstorming):* Свободно, нефилтрирано генериране на идеи без първоначална критика или оценка. Тази техника стимулира креативността и позволява откриването на иновативни решения и нетривиални изисквания.
- **Етнографски анализ:** Това е техника, заимствана от социалните науки, която включва продължително наблюдение на реалната ежедневна работа на потребителите в тяхната естествена среда. Етнографският анализ позволява откриването на имплицитни (неосъзнати) изисквания, които потребителите не могат да формулират директно, тъй като те са станали част от техните автоматизирани работни навици. Тази техника е особено ценна при сложни и дълбоко вкоренени бизнес процеси.
- **Прототипиране:** Тази техника включва създаване на ранни, опростени версии на потребителските интерфейси или функционални компоненти на системата. Прототипите могат да бъдат:
 - *Ниско-фиделитетни:* Хартиени скици или екрани с ниска детайлност, използвани за ранно валидиране на концепции.
 - *Високо-фиделитетни:* Интерактивни макети с висока степен на визуална и функционална реалистичност.

Прототипите помагат на потребителите да визуализират бъдещата функционалност и се използват паралелно както за извличане, така и за валидиране на изискванията.
- **Повторно използване (Reusing):** Тази техника се основава на систематичен анализ и адаптиране на изисквания от успешни предходни системи в същия бизнес домен или от стандартизирани библиотеки с изисквания. Повторното използване повишава ефективността, намалява грешките и гарантира съответствие с утвърдени индустриални практики.
- **Сценарии, случаи на употреба (Use Cases) и потребителски истории (User Stories):** Тези техники представят функционалността на системата

чрез описание на конкретни взаимодействия между актьори (потребители или външни системи) и самата система. Сценариите описват последователността от стъпки за изпълнение на дадена задача, като акцентират върху целта на потребителя. Use Cases предоставят по-структуриран формат с дефинирани пред- и постусловия, основни и алтернативни потоци. User Stories са кратки, неформални описания на функционалност от гледна точка на потребителя, широко използвани в гъвкавите методологии.

3.2 Техники за анализ

Процесът по пречистване, структуриране и осигуряване на качеството на извлечените изисквания разчита на следните методи:

- **Контролни списъци (Checklists):** Това са структурирани шаблони със систематизирани въпроси за проверка на качеството на всяко изискване. Типични въпроси включват: "Изискването тестваемо ли е? "Има ли дефиниран ясен източник (произход)? "Изискването еднозначно ли е? "Непротиворечиво ли е с останалите изисквания? "Има ли дефинирани приоритет и спешност?".
- **Преговори (Negotiation):** Това е структуриран процес за разрешаване на конфликти между заинтересовани лица с различни или противоположни приоритети. Преговорите могат да включват търсене на компромис, установяване на ясни критерии за вземане на решения, използване на техники за фасилитация и прилагане на обективни методи за оценка на стойността на различните изисквания.
- **Приоритизация:** Това е систематичен процес на подреждане на изискванията според тяхната критичност за бизнеса и тяхното значение за успеха на проекта. Приоритизацията може да се извършва чрез различни методи – точкови системи, класиране, сравнение по двойки или чрез класически схеми като MoSCoW (Must-have, Should-have, Could-have, Won't-have). Този процес е от съществено значение за ефективното разпределение на ресурсите и управление на обхвата на проекта.
- **Моделиране:** Създаването на абстрактни представяния на изискванията чрез полуформални нотации (UML, DFD, E/R диаграми) позволява по-дълбоко разбиране на структурата, функциите и поведението на системата. Моделирането улеснява идентифицирането на пропуски, противоречия и несъответствия в изискванията, като предоставя визуална и систематизирана перспектива върху софтуерния продукт.

3.3 Техники за валидиране

Валидирането на изискванията гарантира, че създадената спецификация коректно отразява нуждите на заинтересованите лица и че разработваният продукт ще има реална стойност за бизнеса.

- **Прототипи:** Предоставянето на работещ макет или интерактивен прототип за реална оценка от страна на клиента и крайните потребители. Това позволява ранно откриване на грешки в разбирането и несъответствия между очакванията на потребителите и спецификацията, като същевременно служи като

ефективно средство за комуникация между техническите и не-техническите заинтересовани лица.

- **Ревюта и инспекции (Inspections/Reviews):** Това е формален, структуриран групов преглед на SRS документа, извършван от екип от архитекти, тестери, анализатори и представители на клиента. Инспекциите следват строг протокол, включващ подготовка, индивидуален преглед, групова среща и корекция на откритите дефекти. Те са един от най-ефективните методи за откриване на грешки в изискванията на ранен етап.
- **Матрица на проследимост (Traceability Matrix):** Този инструмент установява и документира връзките между различните артефакти в проекта – от бизнес изискванията, през софтуерните изисквания, до архитектурните компоненти и тестовите случаи. Матрицата на проследимост позволява двупосочно проследяване: от бизнес изискване към неговите софтуерни реализации и тестове, и обратно – от даден софтуерен компонент към бизнес изискванията, които го обосновават. Това гарантира, че няма "излишни" функции и че всички бизнес нужди са покрити.
- **Валидиране на модели:** Това представлява логическа проверка на полужормалните диаграми и модели, създадени по време на анализа. Валидирането на модели включва проверка за синтактична коректност, семантична непротиворечивост, пълнота на покритието на сценариите и съответствие с дефинираните бизнес правила.
- **Генериране на тестови сценарии:** Този подход се основава на принципа, че всяко коректно формулирано изискване трябва да позволява създаването на обективни тестови случаи за неговата верификация. Разработването на концептуални тестове на базата на изискванията служи като ефективна техника за валидиране – ако дадено изискване не позволява генерирането на тест, то се счита за невалидно, тъй като не предоставя критерии за проверка на своето изпълнение.

4 Специфициране и модели на софтуерните изисквания

Специфицирането на изискванията е процесът на формално записване и представяне на извлечените и анализирани изисквания в структурирана и комуникативна форма. Този процес е от критическо значение, тъй като спецификацията служи като основен документ за комуникация между всички участници в проекта и като правна и техническа основа за последващите фази на разработка, тестване и приемане на системата.

4.1 Нива на документиране

Специфицирането се извършва на три основни нива на абстракция, всяко от които обслужва различни цели и различни аудитории:

1. **Естествен език:** Това е най-достъпната и най-широко използвана форма на документиране. Представлява свободен текст без строги синтактични ограничения, който е лесен за четене и разбиране от всички заинтересовани лица. Основният недостатък на естествения език е присъщата му многозначност и склонност към неед-

нозначни тълкувания, което може да доведе до грешки в разбирането и имплементацията. Този риск нараства с увеличаване на сложността и обема на изискванията.

2. Структуриран запис: Това ниво въвежда ограничения върху естествения език чрез използване на стандартизирани шаблони, таблични формати и предварително дефинирани структури. Структурираният запис включва използването на псевдокод, алгоритмични описания и формализирани секции в документа, които намаляват многозначността и повишават прецизността на спецификацията. Този подход балансира между четимостта на естествения език и точността на формалните нотации.

3. Модели (полу-формални): Това ниво включва използването на графични езици за моделиране, които предоставят визуално представяне на различни аспекти на системата. Най-често използваните езици за моделиране включват UML (Unified Modeling Language) за обектно-ориентиран анализ и дизайн, DFD (Data Flow Diagrams) за функционално моделиране и E/R (Entity-Relationship) диаграми за моделиране на данни. Полу-формалните модели предоставят стандартизиран начин за представяне на сложни структури и взаимовръзки, като същевременно остават разбираеми за широк кръг от заинтересовани лица.

Дефиниция 1. Перспектива (Perspective) дефинира фокуса на самия модел – какъв конкретен аспект от системата се изследва и анализира. Различните перспективи разкриват различни измерения на системата: нейната статична структура, нейните функции и трансформации на данни, или нейното динамично поведение във времето. Изборът на перспектива определя кои модели са подходящи за дадения контекст на анализ.

Дефиниция 2. Гледна точка (Viewpoint) определя позицията на конкретния наблюдател (актьор, потребител или външна подсистема) спрямо софтуерния продукт. Гледната точка филтрира информацията за системата според интересите и отговорностите на конкретния участник, като представя само тези аспекти, които са релевантни за неговата роля. Например гледната точка на системния администратор се фокусира върху конфигурация, мониторинг и управление на потребителите, докато гледната точка на крайния потребител се фокусира върху функционалност, използваемост и ефективност на взаимодействието.

4.2 Модели в зависимост от перспективата на системата

Изборът на конкретни диаграми и модели се определя от избраната аналитична перспектива, която от своя страна зависи от целите на анализа, фазата на проекта и специфичните изисквания на заинтересованите лица. Трите основни перспективи предоставят взаимодопълващи се гледни точки към системата, като всяка от тях разкрива различен аспект от нейната същност.

4.2.1 1. Перспектива "Данни" (Data perspective)

Тази перспектива се фокусира върху статичната структура на информацията, която системата обработва, съхранява и управлява. Моделите от тази перспектива описват информационните обекти в системата, техните атрибути и връзките между тях, без да се фокусират върху динамиката на тяхната обработка. Тя е фундаментална за разбирането на информационната архитектура на системата.

- **Entity-Relationship (E/R) диаграми:** Това е концептуален модел на данните, който представя системата чрез същности (обекти от реалния свят), атрибути (характеристики на същностите) и връзки (взаимодействия и зависимости между същностите). E/R диаграмите са независими от конкретната технология за съхранение на данни и служат като мост между бизнес разбирането на информацията и нейната техническа реализация.
- **Обектно-ориентирани (ООР) модели:** Тези модели, представени най-често чрез класови диаграми (Class Diagrams) в UML, описват типовете обекти в системата, техните капсулирани данни (атрибути), тяхното поведение (методи) и йерархичните и асоциативни връзки между тях (наследяване, композиция, агрегация). Класовите диаграми са централният модел в обектно-ориентирания анализ и дизайн.
- **Релационни модели:** Това са логически схеми на бази данни, които дефинират реалните таблици, техните колони, типовете данни, както и първичните и външните ключове, които установяват релациите между таблиците. Релационните модели са пряко свързани с физическата реализация на съхранението на данни и служат като основа за имплементация на базата данни.

4.2.2 2. Перспектива "Функции"(Functional perspective)

Тази перспектива изследва трансформацията на данни и процесите, които софтуерът изпълнява. Моделите от тази перспектива описват как входните данни се преобразуват в изходни резултати чрез поредица от функционални стъпки, кои компоненти извършват тези трансформации и в каква последователност.

- **Dataflow диаграми (DFD):** Тези диаграми визуализират движението на потоците от данни през системата, като представят процесите (трансформации на данни), външните субекти (източници и получатели на данни извън системата) и физическите хранилища на данни. DFD диаграмите са йерархични и могат да бъдат детайлизирани на различни нива на абстракция, което ги прави подходящи за анализ на сложни системи.
- **Диаграми на дейностите (Activity Diagrams):** Тези диаграми описват последователния или паралелен работен процес (workflow) на дадена функция, като представят поредицата от действия, условията за разклонение и синхронизация на паралелни потоци. Диаграмите на дейностите са особено полезни за моделиране на бизнес процеси и алгоритмична логика.
- **Диаграми на последователността (Sequence Diagrams):** Тези диаграми моделират времевата последователност на съобщенията, обменяни между обектите при изпълнение на конкретен сценарий. Те фокусират върху хронологията на взаимодействията и са особено полезни за детайлизиране на динамиката на обектно-ориентирани системи и за валидиране на сценариите за употреба.

4.2.3 3. Перспектива "Поведение"(Behavioral perspective)

Тази перспектива анализира динамичните реакции на системата при възникване на вътрешни или външни събития. Моделите от тази перспектива се фокусират върху

това как системата или нейните компоненти променят своето състояние в отговор на различни стимули и какви действия се задействат при тези промени.

- **Диаграма на състоянията (Statechart / State Machine Diagram):** Този модел показва жизнения цикъл на даден обект или на цялата система, представен като краен брой състояния, през които обектът преминава под въздействието на тригери (събития). Диаграмата на състоянията дефинира допустимите преходи между състоянията, условията за тези преходи и действията, които се извършват при влизане или излизане от дадено състояние. Тя е особено полезна за системи с дискретни състояния и сложна логика на преходи.
- **Автомати (Finite State Machines):** Това е математически прецизен формализъм, който се използва за дефиниране на поведението на системи на **ниско ниво на абстракция**. Автоматите намират приложение в дефинирането на комуникационни протоколи, софтуерни драйвери, логически контролери, парсери и други компоненти, където е необходимо строго формално описание на поведението, което да позволява верификация и автоматично генериране на код. В контекста на изискванията, автоматите се използват за прецизно специфициране на критични за безопасността или сигурността компоненти.